Overlap and Extrapolation Notes

Srikar Katta

Contents

1 Overlap and Extrapolation

1.1 Introduction

Much of data science is about prediction: given a unit's covariates, what do we expect its outcome to be? Causal inference asks a harder question: what *would* the outcome have been if we had intervened and changed the unit's treatment? Answering "what if" questions requires more than a good predictive model; it requires assumptions about how treatments were assigned in the first place. Throughout these notes, we will focus on one of those assumptions—*overlap*, also known as *positivity*—and what we can do when it fails. The content here combines the lecture material with the accompanying lab, so we will move back and forth between the statistical ideas and Python code that implements them.

To set up notation, suppose we observe n units. For each unit i, we observe a covariate vector $\mathbf{X}_i \in \mathbb{R}^p$ describing the unit, a binary treatment indicator $T_i \in \{0, 1\}$, and an outcome $Y_i \in \mathbb{R}$.

Definition 0.1: Potential outcomes

Each unit i has two *potential outcomes*: $Y_i(1)$, the outcome unit i would experience if treated, and $Y_i(0)$, the outcome unit i would experience if untreated. We only ever observe one of the two—the one corresponding to the treatment the unit actually received. The unobserved one is called the counterfactual.

The estimand we will care about in these notes is the average treatment effect.

Definition 0.2: Average treatment effect (ATE)

The average treatment effect is the expected difference between a unit's two potential outcomes,

averaged over the population:

$$\tau = \mathbb{E}\left[Y_i(1) - Y_i(0)\right].$$

1.2 The Rubin Causal Model and Its Assumptions

Because we never observe both potential outcomes for the same unit, estimating τ from observational data requires assumptions. The Rubin Causal Model relies on three:

1. **No unobserved confounding.** Conditional on the observed, pre-treatment covariates, the treatment is independent of the potential outcomes: $(Y_i(1), Y_i(0)) \perp T_i \mid \mathbf{X}_i$.
2. **SUTVA.** My observed outcome depends only on my own assigned treatment: $Y_i = T_i Y_i(1) + (1 - T_i)Y_i(0)$. There is no interference between units and no hidden versions of the treatment.
3. **Positivity/overlap.** Everyone has a chance of being assigned to either treatment or control.

The first two assumptions get most of the attention in courses and papers. These notes are about the third.

Definition 0.3: Positivity (overlap)

Positivity holds if every unit has a strictly positive probability of receiving either treatment:

$$0 < \mathbb{P}(T = 1 \mid \mathbf{X} = x) < 1 \quad \text{for all } x \text{ in the support of } \mathbf{X}.$$

Overlap is violated wherever $\mathbb{P}(T = 1 \mid \mathbf{X} = x)$ equals exactly 0 or exactly 1.

1.3 What Goes Wrong Without Overlap

Suppose there is a region of the covariate space where $\mathbb{P}(T_i = 1 \mid \mathbf{X}_i \in \text{region}) = 0$. Then we will *never* observe a treated unit in that region, no matter how much data we collect. What do you think happens when we try to estimate $\mathbb{E}[Y(1) \mid \mathbf{X}]$ there?

We have two unappealing options. The first is to *extrapolate*: fit a parametric model for the treated outcome using data from outside the region and trust the model's predictions inside it. The problem is that we have no way of knowing whether the parametric assumption is right—there is, by construction, no treated data in the region against which to check it. A linear model and a quadratic model may agree beautifully where we have data and disagree wildly where we do not, and the data cannot adjudicate between them. The second option is to give up on that region entirely, which brings us to a new estimand.

1.4 Moving the Goalpost: The ATE for the Overlap Region

If we cannot say anything credible about the no-overlap region, we can simply remove it from the question we are asking.

Definition 0.4: Average treatment effect for the overlap region (ATEO)

The average treatment effect for the overlap region restricts the ATE to units in regions of the

covariate space where overlap holds:

$$\tau^O = \mathbb{E}\left[Y_i(1) - Y_i(0) \mid \mathbf{X}_i \in \text{overlap region}\right].$$

This is an honest fix, but be aware of what it costs us: we are "moving the goalpost." Conclusions now apply only to the overlap population, not the overall population, and in practice we might not even know exactly who is in the overlap population. Removing the bad region is really easy when we have one or two dimensions—just visual inspection works. But what happens with 3 dimensions? Or 4? Or 20? Or 500? We need automated methods. There are many; we will discuss three very simple ones: propensity score trimming, overlap weights, and matching.

1.5 Propensity Score Trimming

All three methods revolve around a single object: the propensity score.

Definition 0.5: Propensity score

The propensity score is the probability that a unit with covariates x is treated:

$$\pi(x) = \mathbb{P}(T = 1 \mid \mathbf{X} = x).$$

By definition, overlap is violated exactly where $\pi(x) = 0$ or $\pi(x) = 1$.

If we knew the true propensity score, finding positivity violations would be trivial: flag every unit with $\pi(\mathbf{X}_i)$ equal to (or extremely close to) 0 or 1. Of course, we do not know the true propensity score, so we estimate it, and we keep only units whose *estimated* score is comfortably in the interior of $[0, 1]$. The full procedure is:

1. Split the data into a training fold and an estimation fold. The split protects us from overfitting when we use flexible machine learning methods to fit the propensity score.
2. On the training fold, learn an estimator that predicts the observed treatment given the observed covariates. Do not use a binary classifier that fails to offer probabilistic predictions—we need $\widehat{\pi}(x)$, not just a predicted label.
3. Use the model from Step 2 to estimate propensity scores on the estimation fold.
4. Remove any unit in the estimation fold with a really high or really low estimated propensity score. A common rule of thumb is to keep only units with $0.1 \leq \widehat{\pi}(\mathbf{X}_i) \leq 0.9$.
5. Estimate treatment effects on the remaining data in the estimation fold.

In code, the trimmer is only a few lines. It takes any scikit-learn compatible classifier as the propensity score estimator:

```
def ps_trimming(X_train, X_test, T_train, ps_estimator):
    ## Step 1: estimate propensity scores on held-out data
    ps_hat = ps_estimator.fit(X_train, T_train).predict_proba(X_test)[:, 1]
    ## Step 2: keep observations with scores between 0.1 and 0.9
    keep_idx = (ps_hat >= 0.1) & (ps_hat <= 0.9)
    ## Step 3: return trim ID
    return keep_idx
```

Two warnings. First, trimming depends a lot on the choice of thresholds; $[0.1, 0.9]$ is a convention, not a law of nature. Second—and we will see this vividly in the simulation—trimming depends a lot on having the right propensity score estimator. A model that cannot represent the violation cannot find it.

1.6 Overlap Weights

Trimming makes a hard, all-or-nothing decision about each unit. Weighting is the soft version of the same idea: upweight "really good" observations and downweight "really bad" ones. To see where the weights come from, recall inverse propensity weighting (IPW):

$$\widehat{\tau} = \frac{1}{N}\sum_{i=1}^{N}\frac{T_i Y_i}{\pi(\mathbf{x}_i)} - \frac{1}{N}\sum_{i=1}^{N}\frac{(1-T_i)Y_i}{1-\pi(\mathbf{x}_i)}.$$

IPW upweights treated and control units that are underrepresented in their assignment group. That is exactly the wrong behavior near a positivity violation: a treated unit with $\pi(\mathbf{x}_i) \approx 0$ receives an enormous weight, and the estimator's variance explodes.

Overlap weights flip the logic: instead of upweighting rare units, we upweight units in regions of *high* overlap—units that plausibly could have received the other treatment. The weight for each unit is the probability of being assigned to the *opposite* treatment group:

$$\text{weight for treated unit } i: \quad \frac{1-\pi(\mathbf{x}_i)}{\sum_{j=1}^{N} T_j\,[1-\pi(\mathbf{x}_j)]}, \qquad \text{weight for control unit } i: \quad \frac{\pi(\mathbf{x}_i)}{\sum_{j=1}^{N}(1-T_j)\,\pi(\mathbf{x}_j)}.$$

A treated unit with $\pi(\mathbf{x}_i)$ near 1 (almost surely treated, no comparable controls) gets weight near 0; a unit with $\pi(\mathbf{x}_i)$ near 0.5 gets the largest weight. The pipeline is identical to trimming—split, train a propensity score estimator, estimate scores on the estimation fold—except that instead of removing observations, we weight them before feeding them into the ATE estimator. Like trimming, the method lives or dies by the quality of the estimated propensity score.

1.7 Handling Overlap Violations by Matching

Propensity score methods reduce the entire covariate space to one number. Matching instead asks a geometric question: for each unit, how far do I have to search to find a nearest neighbor of the *opposite* treatment group? If a treated unit's nearest control is very far away, that unit lives in a region with no comparable controls—an overlap violation. The algorithm is:

1. Specify a distance metric d_M in the covariate space, e.g., Euclidean (L_2) or L_1 distance.
2. For each unit, find the distance to its (k-)nearest neighbor in the opposite treatment group.
3. Use outlier detection on those distances to flag units with an anomalous nearest-neighbor distance.

In the lab, we flag any unit whose opposite-group nearest-neighbor distance exceeds the classic boxplot threshold, the 75th percentile plus 1.5 times the interquartile range:

```
def matching_trimmer(X_test, T_test):
    ## Step 1: separate treated and control units
    X_tx, X_ctrl = X_test[T_test == 1], X_test[T_test == 0]
    ## Steps 2-3: distance to nearest treated / nearest control neighbor
    tx_dist = NearestNeighbors(n_neighbors=1).fit(X_tx).kneighbors(X_test)[0][:, 0]
```

```
    ctrl_dist = NearestNeighbors(n_neighbors=1).fit(X_ctrl).kneighbors(X_test)[0][:, 0]
    ## Step 4: keep only the distance to the _opposite_ class
    distances = tx_dist * (1 - T_test) + ctrl_dist * T_test
    ## Step 5: outlier threshold via interquartile range
    q75, q25 = np.percentile(distances, [75, 25])
    outlier_threshold = q75 + 1.5 * (q75 - q25)
    ## Step 6: keep only non-outliers
    return distances < outlier_threshold
```

What is similar and what is different between this and propensity-score-based methods? Both try to locate regions where one treatment group is missing, and both depend on a modeling choice—the propensity score estimator there, the distance metric here. The big difference is *auditability*. Matching produces, for each questionable unit, a concrete set of nearest neighbors that a human can inspect. Consider a study of some treatment's effect on diabetic shock, where "hypo" indicates a past diabetic shock. Suppose the lone control unit has hypo = True while every treated unit in its matched neighborhood has hypo = False, with the other covariates (age, degree, race, HbA1c, years, sex) nearly identical. There is no overlap between treated and control on hypo, and the matched table makes that visible: we can directly ask a domain expert whether extrapolating across the hypo variable is justifiable. For a clinically crucial variable like a history of diabetic shock, we definitely cannot justify it (Katta et al., 2024, AISTATS). Propensity score methods would quietly fold that variable into a single number; matching lets us validate whether extrapolation is defensible, covariate by covariate.

1.8 A Simulation Study

We now evaluate these ideas in a setting where we control the truth. The advantage of simulation is that we generate our own data, so ground-truth treatment effects are known and we can measure each estimator's bias and variance exactly. We use the following data generating process. For each $i = 1, \ldots, n$:

- Sample the covariate vector: $\mathbf{X}_i \sim \text{Unif}[-1, 1]^p$ with $p = 2$ covariates.
- Assign treatment: $T_i = 1$ with probability $\text{expit}(0.5\,\mathbf{X}_{i,1} + 0.5\,\mathbf{X}_{i,2})$.
- Generate outcomes: $Y_i = \mu(\mathbf{X}_i, T_i) + \epsilon_i$, with $\epsilon_i \sim \mathcal{N}(0, 1)$ and

$$\mu(\mathbf{X}_i, T_i) = 10 + \mathbf{X}_{i,1} + 2\mathbf{X}_{i,2} + 10\,T_i + \underbrace{10\,\mathbf{X}_{i,2}^2\,T_i}_{\text{tx effect hetero.}}.$$

```
def expit(x):
    return 1 / (1 + np.exp(-x))

def make_pi(X):
    return expit(0.5 * X[:, 0] + 0.5 * X[:, 1])

def mu(X, T):
    return 10 + X[:, 0] + 2 * X[:, 1] + 10 * T + 10 * X[:, 1]**2 * T
```

The conditional average treatment effect is $\mu(x, 1) - \mu(x, 0) = 10 + 10x_2^2$, so since $\mathbb{E}[X_2^2] = 1/3$ for $X_2 \sim \text{Unif}[-1, 1]$, the true ATE is $10 + 10/3 \approx 13.33$. To violate positivity, we introduce a new propensity score: any unit in the bottom-left corner of the covariate space ($\mathbf{X}_{i,1} \leq -0.5$ and

$\mathbf{X}_{i,2} \leq -0.5$) is *guaranteed* to be in the control group,

$$\pi^{\text{viol}}(\mathbf{x}) = \text{expit}(0.5x_1 + 0.5x_2) \cdot [\mathbf{1}_{x_1 \geq -0.5} + \mathbf{1}_{x_2 \geq -0.5}],$$

and the ATEO conditions on being outside that corner: $\tau^O = \mathbb{E}[Y(1) - Y(0) \mid \mathbf{X}_{i,1} \geq -0.5 \text{ or } \mathbf{X}_{i,2} \geq -0.5] \approx 13.17$. Figure 1 plots the two propensity score functions; the dark purple square in the right panel is the dead zone where $\pi(x) = 0$.

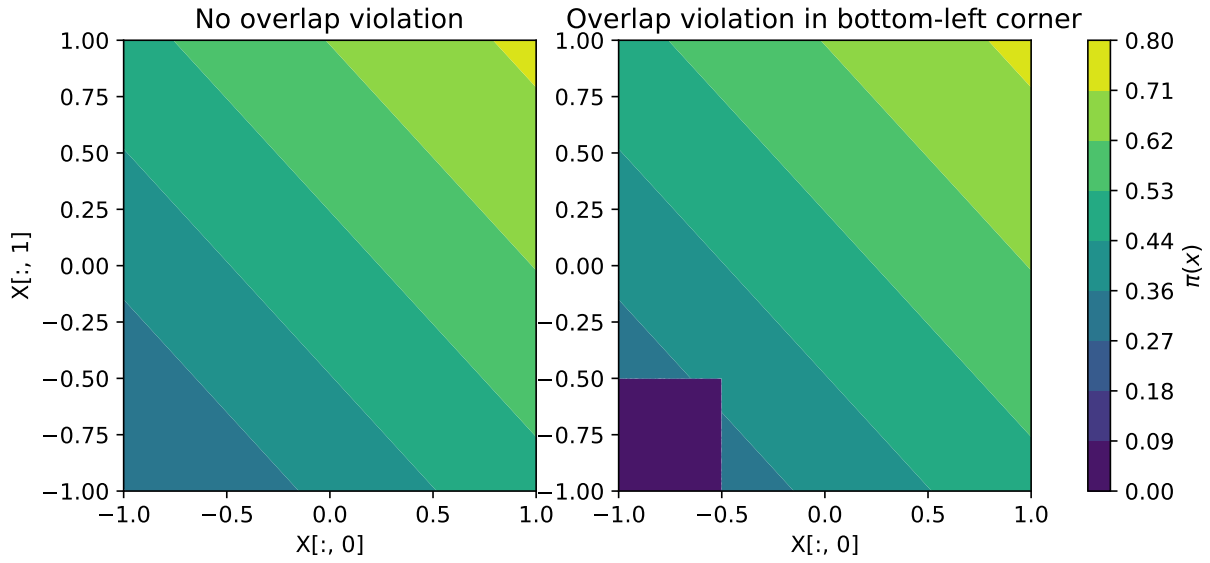

Figure 1: True propensity scores without (left) and with (right) the positivity violation.

We run 100 Monte Carlo iterations: each iteration samples $n = 10{,}000$ observations, then estimates the ATE with two estimators—IPW with a logistic regression propensity score, and an outcome regression "T-learner" that fits a separate linear regression to each treatment arm and averages the difference in predictions. Without the violation, both estimators behave well: the IPW estimator has bias ≈ 0.000 and the T-learner bias ≈ -0.053 (a small misspecification penalty, since the true outcome model is quadratic in x_2 for treated units), with variances around 0.003–0.004. With the violation, both fall apart, as shown in Figure 2. Neither sampling distribution covers the true ATE (red dashed line): the IPW estimator's bias relative to the ATE is -0.50 and the T-learner's is -0.27. Notice that they are *also* biased for the ATEO (black dashed line), each in its own way: the logistic propensity score cannot represent a hard zero, and the linear outcome model extrapolates incorrectly into the corner where it has no treated data. This is the practical lesson: even knowing the *true* propensity score would not rescue the ATE, because the data simply contain no information about $Y(1)$ in the corner. The ATE for the full population is not identified; the ATEO is the best we can target.

Next we implement propensity score trimming with two estimators—a logistic regression and a random forest—alongside the matching trimmer, and compare which observations each one prunes on a single draw of 2,500 observations (Figure 3). The differences are striking. The logistic regression trimmer drops 0 of the 1,250 estimation-fold units: a logistic model is smooth and monotone in a linear index, so it underfits and literally cannot produce estimated scores near 0 in the corner. The random forest drops 140 units, slightly more than the 72 truly violating units—it correctly carves out the corner but, being flexible, also overfits a few noisy pockets elsewhere. The matching

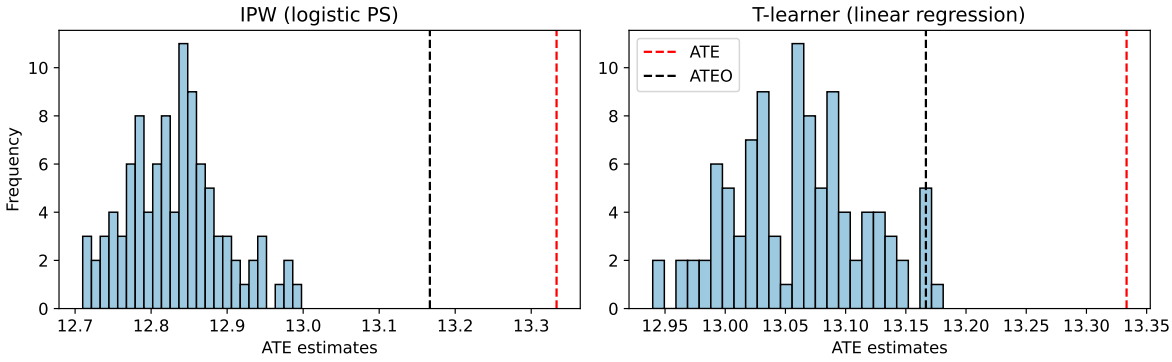

Figure 2: Monte Carlo distributions of untrimmed ATE estimates under the positivity violation.

trimmer drops 69 units, almost exactly the right set, by noticing that units deep in the corner must search far to find a treated neighbor. The general pattern when answering "logit vs. random forest": if the true assignment mechanism is simple and smooth, logistic regression's stability wins; if the mechanism has sharp or complex structure—like our hard corner—only a flexible learner can detect it, at the cost of some overfitting.

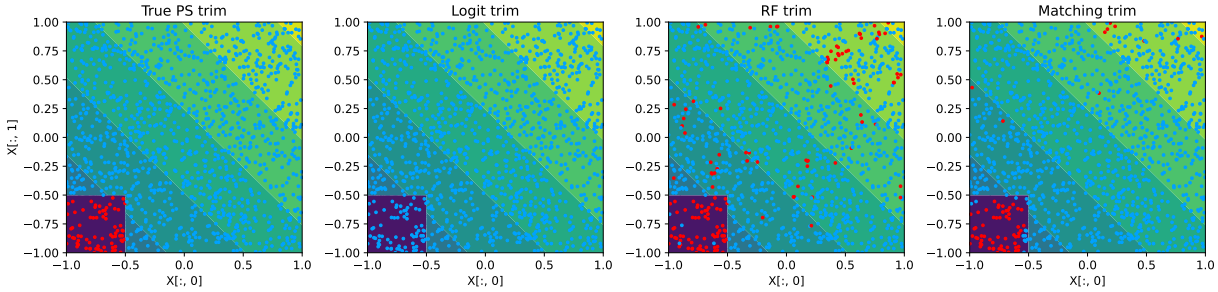

Figure 3: Kept (blue) and pruned (red) units under each trimmer, over the true propensity score.

Finally, we rerun the Monte Carlo experiment, this time trimming before estimating (Figure 4). Because logistic trimming removes nothing, its downstream T-learner estimates remain biased for the ATEO (bias -0.11). Random forest trimming, in contrast, essentially solves the problem: the trimmed IPW estimator has bias -0.04 and the trimmed T-learner -0.07 relative to the ATEO, with the remaining gap coming from the slight over-pruning we saw above. Note also that no amount of trimming brings us back to the red ATE line—trimming changes the estimand, not the missing data.

1.9 Choosing an Approach

There are a lot of different approaches to handling positivity violations, and each has its own benefits. Trimming and overlap weights depend a lot on the propensity score estimator; matching depends a lot on the choice of distance metric—but lets us double check the results by inspecting matched units directly. A general rule of thumb: if you think your treatment assignment mechanism is simpler than the outcome generation function, go with trimming or overlap weights. For example, if a doctor only looks at heart rate and blood pressure when deciding whether to prescribe blood pressure medication, the assignment mechanism is low-dimensional and easy to model even when health outcomes are not.

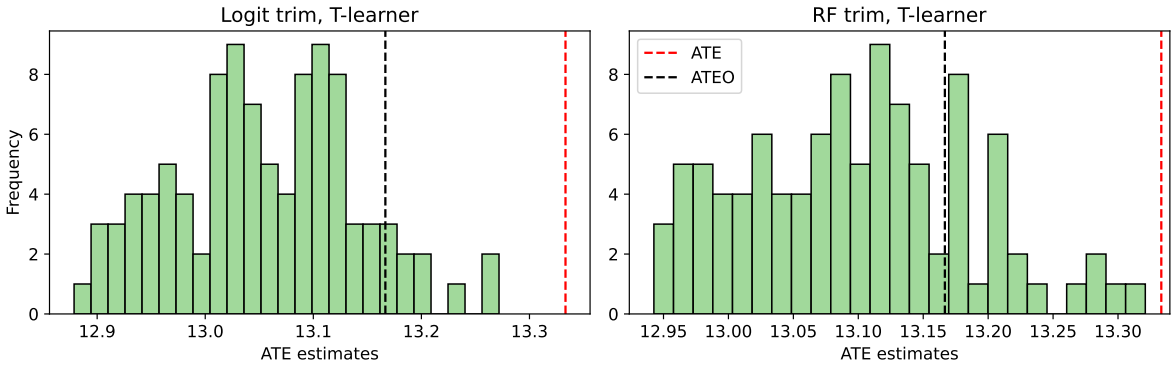

Figure 4: Monte Carlo distributions of T-learner estimates after logistic (left) and random forest (right) propensity score trimming. Only the random forest trimmer concentrates around the ATEO.

If a new researcher came to us suspecting a positivity violation, our recommendations would be: (1) plot or otherwise probe the estimated propensity score distribution and nearest-neighbor distances before estimating anything; (2) use a propensity score estimator flexible enough to actually represent the suspected violation, validated on held-out data; (3) report the ATEO honestly as the estimand, alongside who was pruned; and (4) where possible, audit the pruned region with matching so a domain expert can judge whether any extrapolation is defensible.

Two issues haunt all of these approaches. First, we are moving the goalpost—we cannot make claims about the overall population, only the overlap population. Second, we might not know exactly who is in the overlap population, since our trimmers and weights are themselves estimated. Handling overlap violations is therefore less about finding the one correct method and more about being honest about what question the data can actually answer.